

**LAPORAN**  
**MATA KULIAH PENGENALAN POLA**



**Disusun Oleh:**

**Pramitha Witawacita Astadewi      24/554318/NPA/19979**

**Najma Clara Bella      24/554317/NPA/19978**

**DEPARTEMEN ELEKTRONIKA DAN  
INSTRUMENTASI FAKULTAS MATEMATIKA DAN  
ILMU PENGETAHUAN ALAM UNIVERSITAS  
GADJAH MADA**

**YOGYAKARTA  
2025**

## ABSTRAK

Peningkatan jumlah publikasi ilmiah di era digital menghadirkan tantangan dalam pengelolaan informasi, terutama dalam mengukur kemiripan antar-dokumen secara otomatis. Penelitian ini bertujuan untuk mengukur kemiripan antar-abstrak artikel ilmiah menggunakan metode Term Frequency-Inverse Document Frequency (TF-IDF) dan cosine similarity. Tiga abstrak artikel dari International Journal of Computing and Cybernetics Systems (IJCCS) dianalisis sebagai data uji. Proses dimulai dengan pra pemrosesan teks untuk menghapus stopwords dan normalisasi teks. Selanjutnya, nilai TF-IDF dihitung untuk setiap term dalam abstrak, yang kemudian digunakan sebagai vektor representasi teks. Pengukuran kemiripan dilakukan menggunakan cosine similarity yang menghitung sudut antar-vektor. Hasil analisis menunjukkan bahwa ketiga abstrak memiliki kemiripan yang sangat rendah dengan nilai cosine similarity di bawah 0,1, menunjukkan perbedaan topik yang signifikan. Metode ini efektif dalam mengukur kemiripan teks secara otomatis dan dapat diterapkan pada berbagai jenis dokumen teks.

*Kata Kunci : TF-IDF, Cosine Similarity, Text Mining, Kemiripan Dokumen, Analisis Teks.*

# PENDAHULUAN

## I. LATAR BELAKANG

Di era digital ini, jumlah publikasi ilmiah terus meningkat di setiap tahunnya. Hal ini menimbulkan kebutuhan akan teknik *text mining* yang mampu mengekstraksi informasi, mengelompokkan dokumen, dan menemukan keterkaitan antar-naskah secara otomatis. Salah satu pendekatan paling umum untuk mengukur kemiripan dokumen adalah kombinasi skema pembobotan **Term Frequency-Inverse absument Frequency (TF-IDF)** dan **ukuran kesamaan kosinus (*cosine similarity*)**. TF-IDF menekankan istilah-istilah yang sering muncul di suatu dokumen tetapi jarang muncul di dokumen lain, sedangkan cosine similarity memetakan dokumen ke ruang vektor dan mengukur sudut di antara vektor-vektor tersebut sebagai derajat kemiripan.

## II. RUMUSAN MASALAH

1. Bagaimana mengukur kemiripan antar-abstrak artikel ilmiah secara otomatis menggunakan metode text mining?
2. Bagaimana memanfaatkan metode TF-IDF dan cosine similarity untuk menganalisis kemiripan antar-abstrak artikel ilmiah?

## III. TUJUAN

1. Mengembangkan metode untuk mengukur kemiripan antar-abstrak artikel ilmiah secara otomatis menggunakan text mining.
2. Menerapkan metode TF-IDF dan cosine similarity untuk menganalisis kemiripan antar-abstrak artikel ilmiah.

## IV. MANFAAT

1. Mempermudah proses pengelompokan dan analisis dokumen ilmiah secara otomatis.
2. Memberikan pemahaman tentang penerapan metode text mining, khususnya TF-IDF dan cosine similarity, pada dokumen ilmiah.

## HASIL DAN PEMBAHASAN

### I. JURNAL YANG DIGUNAKAN

| No | Judul Jurnal                                                                                                         | Link Jurnal                                                                                                         |
|----|----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| 1  | Exploring the Relationship Between Artificial Intelligence and Business Performance                                  | <a href="https://jurnal.ugm.ac.id/ijccs/article/view/86697">https://jurnal.ugm.ac.id/ijccs/article/view/86697</a>   |
| 2  | Enhancing Neural Collaborative Filtering with Metadata for Book Recommender System                                   | <a href="https://jurnal.ugm.ac.id/ijccs/article/view/103611">https://jurnal.ugm.ac.id/ijccs/article/view/103611</a> |
| 3  | Utilizing Machine Learning for Pattern Recognition of Wayang Kamasan in Efforts to Digitize Traditional Balinese Art | <a href="https://jurnal.ugm.ac.id/ijccs/article/view/102313">https://jurnal.ugm.ac.id/ijccs/article/view/102313</a> |

### II. DATA YANG DIDAPAT

| Exploring the Relationship Between Artificial Intelligence and Business Performance |                |                  |
|-------------------------------------------------------------------------------------|----------------|------------------|
| No                                                                                  | Kata           | Jumlah Frekuensi |
| 1                                                                                   | ai             | 10 Kali          |
| 2                                                                                   | business       | 8 Kali           |
| 3                                                                                   | adoption       | 7 Kali           |
| 4                                                                                   | performance    | 6 Kali           |
| 5                                                                                   | relationship   | 3 Kali           |
| 6                                                                                   | impact         | 3 Kali           |
| 7                                                                                   | implementation | 3 Kali           |
| 8                                                                                   | factors        | 3 Kali           |
| 9                                                                                   | challenge      | 2 Kali           |
| 10                                                                                  | also           | 1 Kali           |

| Enhancing Neural Collaborative Filtering with Metadata for Book Recommender System |       |                  |
|------------------------------------------------------------------------------------|-------|------------------|
| No                                                                                 | Kata  | Jumlah Frekuensi |
| 1                                                                                  | fencp | 6 Kali           |
| 2                                                                                  | book  | 5 Kali           |

|    |            |        |
|----|------------|--------|
| 3  | cold       | 4 Kali |
| 4  | item       | 4 Kali |
| 5  | start      | 4 Kali |
| 6  | commerce   | 2 Kali |
| 7  | data       | 2 Kali |
| 8  | rating     | 2 Kali |
| 9  | challenges | 1 Kali |
| 10 | also       | 2 Kali |

| Utilizing Machine Learning for Pattern Recognition of Wayang Kamasan in Efforts to Digitize Traditional Balinese Art |          |                  |
|----------------------------------------------------------------------------------------------------------------------|----------|------------------|
| No                                                                                                                   | Kata     | Jumlah Frekuensi |
| 1                                                                                                                    | wayang   | 6 Kali           |
| 2                                                                                                                    | culture  | 5 Kali           |
| 3                                                                                                                    | local    | 5 Kali           |
| 4                                                                                                                    | kamasan  | 5 Kali           |
| 5                                                                                                                    | cultural | 4 Kali           |
| 6                                                                                                                    | balinese | 3 Kali           |
| 7                                                                                                                    | approach | 3 Kali           |
| 8                                                                                                                    | machine  | 3 Kali           |
| 9                                                                                                                    | learning | 3 Kali           |
| 10                                                                                                                   | also     | 1 Kali           |

### III. PERHITUNGAN TF-IDF

#### 1. Daftar Term (Kata) yang Digunakan

ai, business, adoption, performance, relationship, impact, implementation , factors, artificial, challenges, fencp, book, cold, item, start, commerce, data, rating, sales, also, wayang, culture, local, kamasan, cultural, balinese, approach, machine, learning, preservation

Total term unik: **30**

Menghitung Frekuensi kata / term

Frekuensi mengindikasikan seberapa sering suatu kata muncul dalam setiap dokumen.

Contoh:

- Kata **challenges** tercatat sebanyak 2 kali dalam abs 1, 2 kali dalam abs2, dan tidak muncul di abs3.
- Kata **also** ditemukan 1 kali dalam abs 1, 2 kali dalam abs2, 2 kali dalam abs 3

#### 2. Perhitungan TF (Term Frequency)

- a. Hitung total token (semua kemunculan term)

Total=10+8+7+6+3+3+3+3+2+1=46 kata

- b. Rumus TF

$$TF(\text{term}) = \frac{\text{Frekuensi}(\text{term})}{\text{Total token dokumen}} = \frac{f_t}{46}$$

- c. Tabel TF lengkap

| No | Term         | Frekuensi (ft) | TF = ft / 46 | TF (dalam %) |
|----|--------------|----------------|--------------|--------------|
| 1  | ai           | 10             | 0,2174       | 21,74 %      |
| 2  | business     | 8              | 0,1739       | 17,39 %      |
| 3  | adoption     | 7              | 0,1522       | 15,22 %      |
| 4  | performance  | 6              | 0,1304       | 13,04 %      |
| 5  | relationship | 3              | 0,0652       | 6,52 %       |

|    |                |   |        |        |
|----|----------------|---|--------|--------|
| 6  | impact         | 3 | 0,0652 | 6,52 % |
| 7  | implementation | 3 | 0,0652 | 6,52 % |
| 8  | factors        | 3 | 0,0652 | 6,52 % |
| 9  | challenge      | 2 | 0,0435 | 4,35 % |
| 10 | also           | 1 | 0,0217 | 2,17 % |

### 3. Perhitungan DF ( absument Frequency )

| Term           | DF | Keterangan              |
|----------------|----|-------------------------|
| ai             | 10 | Muncul hanya di abs1    |
| business       | 8  | Muncul hanya di abs1    |
| adoption       | 7  | Muncul hanya di abs1    |
| performance    | 6  | Muncul hanya di abs1    |
| relationship   | 3  | Muncul hanya di abs1    |
| impact         | 3  | Muncul hanya di abs1    |
| implementation | 3  | Muncul hanya di abs1    |
| factors        | 3  | Muncul hanya di abs1    |
| artificial     | 2  | Muncul hanya di abs1    |
| challenges     | 2  | Muncul di abs1 dan abs2 |
| fencp          | 6  | Muncul hanya di abs2    |
| book           | 5  | Muncul hanya di abs2    |
| cold           | 4  | Muncul hanya di abs2    |
| item           | 4  | Muncul hanya di abs2    |
| start          | 4  | Muncul hanya di abs2    |
| commerce       | 2  | Muncul hanya di abs2    |

|         |   |                                             |
|---------|---|---------------------------------------------|
| data    | 2 | Muncul hanya di abs2                        |
| rating  | 2 | Muncul hanya di abs2                        |
| sales   | 2 | Muncul hanya di abs2                        |
| also    | 2 | Muncul di ketiga abstrak (abs1, abs2, abs3) |
| wayang  | 6 | Muncul hanya di abs3                        |
| culture | 5 | Muncul hanya di abs3                        |

| Term         | DF | Keterangan           |
|--------------|----|----------------------|
| local        | 5  | Muncul hanya di abs3 |
| kamasan      | 5  | Muncul hanya di abs3 |
| cultural     | 4  | Muncul hanya di abs3 |
| balinese     | 3  | Muncul hanya di abs3 |
| approach     | 3  | Muncul hanya di abs3 |
| machine      | 3  | Muncul hanya di abs3 |
| learning     | 3  | Muncul hanya di abs3 |
| preservation | 3  | Muncul hanya di abs3 |

#### 4. Perhitungan IDF (Inverse absument Frequency)

##### a. Rumus

$$IDF(\text{term}) = \log\left(\frac{N + 1}{df(\text{term}) + 1}\right) + 1$$

Dengan:

- N = jumlah dokumen (3) (jumlah dokumen: abs1, abs2, abs3)
- Smoothing +1+1 pada pembilang & penyebut mencegah pembagian 0 (nol)

contoh :

| Term     | df (term) | IDF = $\log((3 + 1)/(df + 1)) + 1$ | Nilai  |
|----------|-----------|------------------------------------|--------|
| ai       | 1         | $\log_{42+1} 24+1$                 | 1,6931 |
| business | 1         | 1,6931                             |        |



|                    |                           |                           |               |
|--------------------|---------------------------|---------------------------|---------------|
| <b>adoption</b>    | 1                         | 1,6931                    |               |
| performance        | 1                         | 1,6931                    |               |
| relationship       | 1                         | 1,6931                    |               |
| impact             | 1                         | 1,6931                    |               |
| implementati<br>on | 1                         | 1,6931                    |               |
| factors            | 1                         | 1,6931                    |               |
| challenge          | 1                         | 1,6931                    |               |
| <b>also</b>        | 2 (ada di abs1<br>& abs2) | $\log 43 + 1 \log 34 + 1$ | <b>1,2877</b> |

- **+df(term)** = banyaknya dokumen tempat term muncul (sesudah normalisasi huruf-kecil & pembersihan tanda baca).
- Semua term di atas hanya muncul di satu dokumen kecuali **also**, yang terdapat di abs1 dan abs2.
- Semakin kecil df  $\Rightarrow$  semakin besar IDF  $\Rightarrow$  term lebih “unik” di korpus.
- Term “ai”, “business”, dll. punya IDF  $\approx 1,69$  karena eksklusif pada satu abstrak.
- Term “also” sedikit lebih umum (IDF  $\approx 1,29$ ) karena muncul di dua abstrak.

## 5. Perhitungan TF-IDF

### a. Rumus

$$TF(\text{term}, \text{dok}) = \frac{f_{t,d}}{\text{Total token dok}}, \quad IDF(\text{term}) = \log\left(\frac{N+1}{df(\text{term})+1}\right) + 1, \quad TF-IDF = TF \times IDF$$

- **df(term)** = banyaknya dokumen yang mengandung term (setelah normalisasi).
- +1 pada IDF untuk smoothing agar tak terjadi pembagian 0.

### b. Hitung IDF :

- Semua term kecuali also hanya ada di 1 dokumen  $\rightarrow df = 1$   
 $IDF = \log 42 + 1 = 1.6931$ 
 $IDF = \log 24 + 1 = 1.6931$
- also muncul di 2 dokumen  $\rightarrow df = 2$   
 $IDF = \log 43 + 1 = 1.2877$ 
 $IDF = \log 34 + 1 = 1.2877$

### c. Table Abs1 contoh :

| Term      | Frekuensi | TF     | IDF    | TF-IDF        |
|-----------|-----------|--------|--------|---------------|
| <b>ai</b> | 10        | 0.2174 | 1.6931 | <b>0.3681</b> |

|                 |   |        |        |        |
|-----------------|---|--------|--------|--------|
| <b>business</b> | 8 | 0.1739 | 1.6931 | 0.2945 |
| <b>adoption</b> | 7 | 0.1522 | 1.6931 | 0.2576 |
| performance     | 6 | 0.1304 | 1.6931 | 0.2208 |
| relationship    | 3 | 0.0652 | 1.6931 | 0.1104 |
| impact          | 3 | 0.0652 | 1.6931 | 0.1104 |
| implementation  | 3 | 0.0652 | 1.6931 | 0.1104 |
| factors         | 3 | 0.0652 | 1.6931 | 0.1104 |
| challenge       | 2 | 0.0435 | 1.6931 | 0.0736 |
| also            | 1 | 0.0217 | 1.2877 | 0.0280 |

- **ai** memiliki skor TF-IDF tertinggi (0,3681) → paling representatif bagi dokumen ini.
- Term lain yang menonjol: **business** dan **adoption**, sejalan dengan fokus abstrak pada adopsi AI di kinerja bisnis.
- **also** diberi bobot rendah karena frekuensi kecil *dan* IDF lebih rendah (muncul di lebih dari satu dokumen), sehingga kontribusinya terhadap kekhasan dokumen kecil.

## 6. Cosine Similarity

adalah cara paling populer untuk mengukur *seberapa mirip* dua objek yang telah direpresentasikan sebagai **vektor angka**—misalnya dokumen teks yang sudah diubah menjadi vektor TF-IDF, profil pengguna dalam sistem rekomendasi, atau fitur citra pada computer-vision.

### a. Intuisi Geometris

Ilustrasi setiap dokumen = vektor di ruang berdimensi-banyak (masing-masing dimensi mewakili satu kata).

- **Panjang vektor** = “seberapa besar” dokumen (banyak kata & bobotnya).
- **Arah vektor** = “komposisi kata” dalam dokumen.

**Cosine similarity** menghitung **cosinus sudut** antara dua vektor A dan B:

$$\cos(\theta) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \|\vec{B}\|}$$

| Nilai | Artinya (vektor)        | Interpretasi Teks                                   |
|-------|-------------------------|-----------------------------------------------------|
| 1     | Sudut 0° (arah identik) | Dokumen identik / sangat mirip                      |
| 0–1   | Sudut kecil             | Mirip (banyak kata tumpang-tindih)                  |
| 0     | Sudut 90° (tegak lurus) | Tidak mirip (hampir tak ada kata sama)              |
| < 0   | Sudut >90°              | Arah berlawanan ( <i>rare</i> —perlu bobot negatif) |

Pada TF-IDF bobotnya non-negatif, jadi rentang praktis 0 – 1.

## 7. Implementasi Program

Pertama-tama, kita memasang dua pustaka inti—`scikit-learn` dan `nlTK`—serta mengunduh daftar stop-words. Langkah ini memastikan Colab memiliki semua modul yang dibutuhkan untuk membuat vektor TF-IDF sekaligus membersihkan teks.

```
✓ [1] !pip install -q scikit-learn nltk
```

```
✓ [2] import nltk, string, os
      nltk.download("stopwords")
```

Begitu lingkungan siap, tiga berkas abstrak—`abs1.txt`, `abs2.txt`, dan `abs3.txt`—diunggah lewat jendela upload Colab.

Setelah berkas-berkas itu berada di direktori kerja, sebuah `Path` loop membaca isi masing-masing file ke dalam daftar `docs_raw`; nama file (tanpa ekstensi) disimpan di `names`. Sebuah pengecekan `assert` memastikan bahwa tepat tiga dokumen berhasil dimuat sehingga proses selanjutnya tidak berjalan dengan dataset yang tidak lengkap.

✓  
17s

```
[3] from google.colab import files
    uploaded = files.upload()
```



Choose Files 3 files

```
abs1.txt(text/plain) - 1537 bytes, last modified: n/a - 100% done
abs3.txt(text/plain) - 1362 bytes, last modified: n/a - 100% done
abs2.txt(text/plain) - 1265 bytes, last modified: n/a - 100% done
Saving abs1.txt to abs1.txt
Saving abs3.txt to abs3.txt
Saving abs2.txt to abs2.txt
```

✓  
0s

```
[4]
from pathlib import Path
docs_raw, names = [], []
for p in sorted(Path('.').glob('abs*.txt')):
    docs_raw.append(p.read_text(encoding='utf-8'))
    names.append(p.stem)
assert len(docs_raw) == 3, "Harus ada 3 file abs1.txt-abs3.txt"
print("Berhasil memuat:", names)
```



Berhasil memuat: ['abs1', 'abs2', 'abs3']

Teks setiap abstrak kemudian dinormalisasi.

Proses normalisasi ini mencakup mengubah huruf menjadi kecil, menghapus tanda baca, serta merapikan spasi ganda.

Fungsi `normalize` dikombinasikan dengan daftar dua bahasa *stopwords* (Inggris dan Indonesia) agar kata-kata umum yang tidak informatif—seperti *the*, *and*, atau *yang*—tidak ikut diperhitungkan.

✓  
0s



```
import re, nltk, string, pandas as pd
from nltk.corpus import stopwords

nltk.download("stopwords", quiet=True)
STOP_WORDS = stopwords.words('english') + stopwords.words('indonesian')
PUNCT_RE = re.compile(r"[^\w\s]", flags=re.UNICODE)

def normalize(text: str) -> str:
    text = text.lower()
    text = PUNCT_RE.sub(" ", text)
    return " ".join(text.split())
```

Selanjutnya, `CountVectorizer` dipakai untuk menghitung frekuensi kemunculan setiap kata setelah tahap pembersihan.

`CountVectorizer` secara otomatis membangun kamus kata dari korpus, menerapkan fungsi `normalize`, dan membuang *stopwords*; hasil akhirnya adalah matriks yang memuat berapa kali masing-masing term muncul di setiap dokumen.

Matriks itu diubah menjadi `DataFrame` agar mudah ditampilkan dan dianalisis.

```

from sklearn.feature_extraction.text import CountVectorizer
import pandas as pd
cv = CountVectorizer(stop_words=STOP_WORDS, preprocessor=normalize)
tf_matrix = cv.fit_transform(docs_raw)

tf_df = pd.DataFrame(
    tf_matrix.toarray().T,
    index=cv.get_feature_names_out(),
    columns=names
)

def top_n(df, col, n=10):
    return df[col].sort_values(ascending=False).head(n)

for col in names:
    display(top_n(tf_df, col).to_frame(name=f"freq_{col}"))

```

Dengan tabel frekuensi di tangan, kita dapat melihat sepuluh kata terpopuler di setiap abstrak—ini berguna sebagai gambaran cepat mengenai topik dominan. Caranya cukup memanggil `top_n(tf_df, "abs1")` dan seterusnya; fungsi tersebut mengambil baris-baris dengan frekuensi terbesar, lalu menampilkannya.

Tahap berikutnya berfokus pada bobot **TF-IDF**. `TfidfVectorizer` tidak hanya menghitung frekuensi lokal suatu kata, tetapi juga mempertimbangkan seberapa jarang kata tersebut muncul di keseluruhan korpus; semakin jarang, semakin tinggi bobotnya. Hasil transformasi disimpan dalam `tfidf_mat`. Kemudian, sebuah fungsi `top_terms` mengekstraksi lima kata dengan bobot TF-IDF tertinggi pada setiap dokumen, sehingga terlihat jelas istilah mana yang paling merepresentasikan masing-masing abstrak.

```

from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd
import numpy as np

tfidf_vectorizer = TfidfVectorizer(stop_words=STOP_WORDS, preprocessor=normalize)
tfidf_mat = tfidf_vectorizer.fit_transform(docs_raw)

terms = np.array(tfidf_vectorizer.get_feature_names_out())

def top_terms(doc_idx, k=10):
    vec = tfidf_mat[doc_idx].toarray().ravel()
    top_ids = vec.argsort()[-k:][::-1]
    return pd.DataFrame({
        "term": terms[top_ids],
        "tfidf_score": vec[top_ids]
    })

print("abs1 - Top 5 term (TF-IDF)")
display(top_terms(names.index("abs1")))

print("abs2 - Top 5 term (TF-IDF)")
display(top_terms(names.index("abs2")))
print("abs3 - Top 5 term (TF-IDF)")
display(top_terms(names.index("abs3")))

```

Kadang kita juga ingin tahu kata apa saja yang sama-sama muncul—dan cukup penting—di dua dokumen sekaligus. Untuk itu dibuatlah fungsi `common_terms`. Ia memilih term dengan bobot lebih besar dari nol di dua vektor TF-IDF, menjumlahkan bobot keduanya sebagai skor gabungan, lalu mengurutkannya dari yang paling besar. Tabel hasilnya membantu menjelaskan mengapa dua abstrak memiliki kemiripan tertentu (atau justru sangat kecil).

```

import pandas as pd
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vectorizer = TfidfVectorizer(stop_words=STOP_WORDS, preprocessor=normalize)
tfidf_vectorizer.fit(docs_raw)

def common_terms(doc_idx_a, doc_idx_b, top_k=10):
    """Mengembalikan DataFrame top_k istilah yang muncul di kedua dokumen."""
    terms = np.array(tfidf_vectorizer.get_feature_names_out())

    vec_a = tfidf_mat[doc_idx_a].toarray().ravel()
    vec_b = tfidf_mat[doc_idx_b].toarray().ravel()

    mask = (vec_a > 0) & (vec_b > 0)
    shared_terms = terms[mask]

    score = (vec_a + vec_b)[mask]

    top = np.argsort(score)[-top_k:][::-1]
    df_shared = pd.DataFrame({
        "term": shared_terms[top],
        f"tfidf_{names[doc_idx_a]}": vec_a[mask][top],
        f"tfidf_{names[doc_idx_b]}": vec_b[mask][top],
    })
    return df_shared

for i in range(len(names)):
    for j in range(i+1, len(names)):
        print(f"\n=== {names[i]} ↔ {names[j]} ===")
        display(common_terms(i, j, top_k=5))

```

Terakhir, keseluruhan kemiripan antar-abstrak diukur lewat **cosine similarity**. `TfidfVectorizer` kembali dipanggil—dengan pra-proses yang sama—untuk memastikan konsistensi bobot. `cosine_similarity` menghitung sudut antara setiap pasangan vektor; nilai 1 berarti identik, sedangkan 0 menandakan tidak ada kemiripan. Setelah angka-angka diperoleh, skrip menambahkan label kualitatif seperti “sedikit mirip” atau “nyaris tak berhubungan” agar pembaca laporan tidak perlu menafsirkan angka mentah

```

[10] from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

vectorizer = TfidfVectorizer(stop_words=STOP_WORDS, preprocessor=normalize)
tfidf_mat = vectorizer.fit_transform(docs_raw)
sim_mat = cosine_similarity(tfidf_mat)

print(sim_mat.round(4))

```

```

[[1.      0.0212 0.019 ]
 [0.0212 1.      0.0093]
 [0.019  0.0093 1.      ]]

```

Pada bagian ini, kami menggunakan loop untuk menghitung **cosine similarity** antara setiap pasangan dokumen (abs1, abs2, abs3). Di dalam loop, nilai kemiripan antara dua dokumen dihitung dan diterjemahkan ke dalam label kualitatif yang menjelaskan tingkat kemiripan. Jika nilai kemiripan (val) lebih besar dari atau sama dengan 0,9, maka akan diberikan label “**Hampir identik**”, untuk nilai di atas 0,7 diberi label “**Sangat mirip**”, dan seterusnya.

Kemudian, hasilnya dicetak dengan format yang mudah dipahami oleh pembaca laporan, sehingga kita bisa melihat nilai kemiripan dalam persentase dan label yang mendeskripsikan seberapa mirip kedua dokumen tersebut.

```
for i in range(3):
    for j in range(i+1, 3):
        val = sim[i, j]
        label = ("Hampir identik" if val >= 0.9 else
                 "Sangat mirip"   if val >= 0.7 else
                 "Cukup mirip"    if val >= 0.4 else
                 "Sedikit mirip"   if val >= 0.1 else
                 "Nyaris tak berhubungan")
        print(f"{names[i]} ↔ {names[j]} : {val:.2%} → {label}")
```

↔ abs1 ↔ abs2 : 2.10% → Nyaris tak berhubungan  
↔ abs1 ↔ abs3 : 1.95% → Nyaris tak berhubungan  
↔ abs2 ↔ abs3 : 0.95% → Nyaris tak berhubungan

Kesimpulan dari hasil ini menunjukkan bahwa ketiga dokumen memiliki kemiripan yang sangat rendah, yang tercermin pada nilai cosine similarity yang lebih rendah dari 0,1, menandakan bahwa ketiganya membahas topik yang sangat berbeda.

## **KESIMPULAN**

Penelitian ini berhasil mengukur kemiripan antar-abstrak artikel ilmiah menggunakan metode Term Frequency-Inverse Document Frequency (TF-IDF) dan cosine similarity. Hasil analisis menunjukkan bahwa ketiga abstrak artikel yang diuji memiliki tingkat kemiripan yang sangat rendah dengan nilai cosine similarity di bawah 0,1. Hal ini mengindikasikan bahwa ketiga abstrak tersebut membahas topik yang berbeda. Metode TF-IDF efektif dalam memberikan bobot pada istilah-istilah penting dalam dokumen, sementara cosine similarity secara akurat mengukur derajat kemiripan antar-dokumen. Metode ini dapat diandalkan sebagai pendekatan text mining untuk mengukur kemiripan teks secara otomatis, serta dapat diterapkan pada analisis dokumen lainnya.